

Lab 05: Validate Product and Customer Data

Maps to: A3, A5

Objective: Define typed product and customer schemas with field-level and cross-field constraints.

Build: schemas.py with ProductCreate, ProductPatch and ProductRead

Tools: Pydantic v2, Swagger UI, OpenAI Python SDK

Lab asset inventory

- ai_vibe.py - OpenAI Python SDK runner with Pydantic structured output and safe generated-file paths
- mock-data.json - authorised synthetic scenario, route contract and test cases
- mock-database.sqlite - inspectable SQLite database seeded only with synthetic commerce and CRM records
- Prompts.pdf - learner-facing first-run, refinement and review prompts
- Prompts.md - copyable source used by the SDK runner
- working-files/ - complete FastAPI, SQLite, HTML/CSS/JavaScript project, including VS Code REST Client requests

AI vibe-coding control loop

Prompts.pdf → ai_vibe.py → Responses API → CodeBundle → working-files/generated/ → pytest + human review

Detailed procedure

Step 1 - Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

1. Perform the action exactly as stated: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 2 - Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

1. Perform the action exactly as stated: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 3 - Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

1. Perform the action exactly as stated: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.
2. Observe the resulting file, HTTP response, log or browser state.

3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 4 - Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

1. Perform the action exactly as stated: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 5 - Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

1. Perform the action exactly as stated: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 6 - Review the generated diff and copy only accepted changes into the working application.

1. Perform the action exactly as stated: Review the generated diff and copy only accepted changes into the working application.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 7 - Define product status as active, inactive or discontinued.

1. Perform the action exactly as stated: Define product status as active, inactive or discontinued.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 8 - Add SKU, product-name, decimal price and stock-quantity constraints.

1. Perform the action exactly as stated: Add SKU, product-name, decimal price and stock-quantity constraints.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 9 - Create separate product and customer input, patch and output models.

1. Perform the action exactly as stated: Create separate product and customer input, patch and output models.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.

4. Save one evidence item before continuing.

Step 10 - Execute valid and invalid requests in /docs.

1. Perform the action exactly as stated: Execute valid and invalid requests in /docs.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 11 - Record each 422 loc, msg and type value.

1. Perform the action exactly as stated: Record each 422 loc, msg and type value.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 12 - Add examples that appear in the generated OpenAPI schema.

1. Perform the action exactly as stated: Add examples that appear in the generated OpenAPI schema.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 13 - Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

1. Perform the action exactly as stated: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Step 14 - Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

1. Perform the action exactly as stated: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.
2. Observe the resulting file, HTTP response, log or browser state.
3. If the expected state is absent, compare the request contract, status code and latest log entry.
4. Save one evidence item before continuing.

Acceptance criteria

- Invalid SKU, price, stock, email and status values are rejected before repository code; valid JSON becomes a typed model.
- Evidence names the lab number and the mapped codes: A3, A5.
- The saved SDK evidence records the prompt, model, assumptions, generated file list and verification result.
- mock-database.sqlite contains synthetic seed data only; no .env, live credential, virtual environment or runtime northstar.db is submitted.

Evidence checklist

- ☐ Prompts.pdf reviewed and dry run captured
- ☐ mock-data.json contains synthetic data only
- ☐ mock-database.sqlite opens in VS Code SQLite Viewer
- ☐ SDK CodeBundle saved under working-files/generated/
- ☐ Generated diff reviewed before applying
- ☐ Working artifact
- ☐ Request/response or test output
- ☐ One failure diagnosis
- ☐ Acceptance criterion confirmed